Evaluation of dependencies

Gabriel Arellano

2026-05-20

Goal and approach

Here we explore dependencies of the (possibly updated) set of functions in the CTFS / fgeo / fgeo2 body of code. One of the problems with the code is the dense network of inter-dependencies. One of the goals of the current evaluation is to reduce dependency to facilitate debugging and improving.

There are relatively large sets of functions that seem to be linked to obsolete functionality, such as pseudo-Bayesian modeling or geometrical code to process digitized paper maps of stems. Identifying those clusters early will facilitate the work, if they are deprecated early.

Because part of the functionality is going to be deprecated, substituted by new code, or delegated into other packages, it is useful to work from high-level functions (they depend on others, but nothing depends on them) to low-level functions (others depend on them, but they don't do much by themselves). If a high-level function gets deprecated, then the dependencies may also be deprecated. Although the order in which the work is being done is not relevant for the user, it can help the current update and future code improvements.

Load functions

First, we declare the path to fgeo2 folder, and source useful auxiliary functions:

```
# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd())
path <- substr(getwd(), 1, g + attr(g, "match.length"))

# Source useful functionality:
source(paste0(path, "evaluation/0-code-to-source.r"))

# Other functionality:
library(igraph, quietly = TRUE)
```

```
Warning: package 'igraph' was built under R version 4.5.2
```

```
Attaching package: 'igraph'
```

```
The following objects are masked from 'package:stats':

    decompose, spectrum
```

```
The following object is masked from 'package:base':

    union
```

This chunk of code will source all the scripts, something similar to the old "startCTFS" function, but following a different structure of folders:

```
f1 <- list.files(paste0(path, "01_utilities/R/new/"), full.names = TRUE)
f2 <- list.files(paste0(path, "02_biomass/R/new/"), full.names = TRUE)
f3 <- list.files(paste0(path, "03_spatial/R/new/"), full.names = TRUE)
f4 <- list.files(paste0(path, "04_abundance_diversity/R/new/"), full.names = TRUE)
f5 <- list.files(paste0(path, "05_demography_growth/R/new/"), full.names = TRUE)
f6 <- list.files(paste0(path, "06_topography_habitat/R/new/"), full.names = TRUE)
f7 <- list.files(paste0(path, "07_mapping_plotting/R/new/"), full.names = TRUE)
ff <- c(f1, f2, f3, f4, f5, f6, f7)
for(f in ff) source(f)
```

Note that it works on (possibly) updated scripts. The history of changes is briefly being recorded in the "function_inventory" table. If you want to evaluate the structure of dependencies in the original code by Condit, change "new" by "old" in the path names above.

Map dependencies

This maps all the mutual dependencies among the whole body of functions:

```
# All functions in the target body of code:
all <- unique(unlist(lapply(ff, function(f) functions_in_script(f))))

# All of the dependencies among all functions in the global environment:
edges <- get_all_dependencies()
```

```
# Subset the dependencies for the target functions only:
keep <- edges$from %in% all | edges$to %in% all
edges <- edges[keep,,drop = FALSE]

# Proper graph representation, maybe useful:
g <- graph_from_data_frame(edges, directed = TRUE)

pdf(paste0(path, "evaluation/results/dependencies-", Sys.Date(), ".pdf"),
    height = 33, width = 33)

plot(g, layout = layout.fruchterman.reingold,
     vertex.label = V(g)$name,
     vertex.size = 5,
     vertex.color= V(g)$color,
     vertex.frame.color= "white",
     vertex.label.color = "black",
     edge.width=E(g)$weight,
     edge.color="black")

dev.off()
```

```
pdf
  2
```

From high-level to low-level

It would be easier to review the code from high to low-level functions:

- High-level functions: nothing depends on them, they depend on others
- Low-level functions: they do not depend on others, they do not do interesting stuff

The reason is to alleviate the body of low-level functions if some of the high-level functions are deprecated or substituted by others with fewer dependencies.

```
# Count number of dependencies and dependents of each function:
n_up <- sapply(all, function(x) length(upstream_dependencies(x, edges)))
n_down <- sapply(all, function(x) length(downstream_dependents(x, edges)))

# Rough rank from high to low level:
how_high <- (n_up / max(n_up)) - (n_down / max(n_down))
as.matrix(head(sort(how_high, decreasing = TRUE), 20))
```

```
                                [,1]
fitSeveralAbundModel       1.0000000
run.growthbin.manyspp      0.9523810
model.littleR.Gibbs        0.8803717
run.growthfit.bin          0.8327526
lmerBayes                  0.7619048
growth.flexbin             0.7131243
binGraphManySpecies.Panel 0.6190476
binGraphManySpecies        0.5470383
compare.growthbinmodel     0.4274100
modelBayes                 0.3809524
abund.manycensus           0.3809524
graph.abundmodel           0.3809524
graph.outliers             0.3809524
pdf.allplot                0.3809524
png.allplot                0.3809524
map2species                0.3809524
full.abundmodel.llike      0.3797909
pt.to.segment              0.3333333
biomass.CTFSdb             0.3333333
fullplot.imageJ            0.3333333
```

Isolated sets of functions

Although most of the code belongs to one network of interdependent code, there are several sets of functions that are disconnected from the rest. Early on, these are potentially low-hanging fruits for review and development. Later, we expect more self-contained modules of intermediate size.

```
# Clusters of functions:
cl <- components(g, mode = "weak")
table(table(cl$membership))
```

```
  2   3   4   7 197
 15   6   1   1   1
```

```
lapply(seq_len(cl$no)[-which.max(cl$csize)], function(i) {
  as.matrix(attr(cl$membership, "names")[cl$membership == i])
})
```

```
[[1]]
     [,1]
[1,] "CalcRingArea"
[2,] "partialcirclearea"
[3,] "circlearea"
[4,] "Annuli"
[5,] "NDcount"
[6,] "NeighborDensities"
[7,] "RipUvK"

[[2]]
     [,1]
[1,] "ba"
[2,] "basum"

[[3]]
     [,1]
[1,] "beta.total"
[2,] "beta.normalized"
[3,] "fit.beta.normal"

[[4]]
     [,1]
[1,] "dbeta.reparam"
[2,] "betaproduct"

[[5]]
     [,1]
[1,] "bootconf"
[2,] "bootstrap.corr"

[[6]]
     [,1]
[1,] "calc.directionslope"
[2,] "calc.gradient"

[[7]]
     [,1]
[1,] "calcS.alpha"
[2,] "d.calcS.alpha"
[3,] "calcalpha"

[[8]]
```

```
     [,1]
[1,] "match.dataframe"
[2,] "CountByGroup"
[3,] "rearrangeSurveyData"

[[9]]
     [,1]
[1,] "order.by.rowcol"
[2,] "order.bynumber"
[3,] "DBHtransition"

[[10]]
     [,1]
[1,] "exponential.sin"
[2,] "dexp.sin"
[3,] "pexp.sin"

[[11]]
     [,1]
[1,] "dgammaMinusdexp"
[2,] "dgammaPlusdexp"
[3,] "rsymexp"
[4,] "dgammadexp"

[[12]]
     [,1]
[1,] "dpois.max"
[2,] "dpois.maxtrunc"

[[13]]
     [,1]
[1,] "segmentPt"
[2,] "drawrectangle"

[[14]]
     [,1]
[1,] "circle"
[2,] "fullcircle"

[[15]]
     [,1]
[1,] "ellipse"
[2,] "fullellipse"
```

```
[[16]]
     [,1]
[1,] "full.xygrid"
[2,] "graph.mvnorm"

[[17]]
     [,1]
[1,] "draw.axes"
[2,] "imageGraph"

[[18]]
     [,1]
[1,] "arrangeParam.llike.2D"
[2,] "lmerBayes.hyperllike.sigma"

[[19]]
     [,1]
[1,] "dmixnorm"
[2,] "minum.mixnorm"

[[20]]
     [,1]
[1,] "qasympower"
[2,] "rasympower"

[[21]]
     [,1]
[1,] "Gibbs.regsigma"
[2,] "Gibbs.regslope"
[3,] "regression.Bayes"

[[22]]
     [,1]
[1,] "se.skewness"
[2,] "se.kurtosis"

[[23]]
     [,1]
[1,] "getTopoLinks"
[2,] "solve.topo"
```